

VIRTUE7 WRITING KIT · OPEN WORKING PAPER

The Compressible Content Architecture, v0.4

A Personal, Governed Runtime for Writing, Research, and
Multi-Format Production

Reference implementation

August 2026

Open source · model neutral · white-label output

FOUNDATION

Abstract

Generative models can produce fluent text quickly, yet fluency alone is a poor foundation for sustained intellectual or creative work. A useful system must preserve the user's purpose, evidence, cadence, decisions, privacy boundaries, and approved form across many sessions and outputs. It must also know which work it can perform from supplied material and which work depends on browsing, retrieval, transcription, analytics, editing, publishing, or other external tools.

The Compressible Content Architecture treats writing as a governed content system rather than a single prompt-response exchange. Its canonical model, $C = (K, U, E, P, Q)$, separates the kernel of meaning, content units, expansion rules, projection contracts, and quality invariants. Version 0.4 adds a personal work layer, $W = (I, V, R, O, S)$, containing authorized identity keys, voice and cadence evidence, role and scope rules, output and Form Lock preferences, and saved commands. Project state and human approvals remain explicit sources of authority.

The result is a portable, model-neutral runtime. It can support fiction, research, creator production, professional documents, legal preparation, product and brand work, company operations, and career materials without forcing those domains into one generic template. Personalization changes routing, evidence standards, structure, cadence, approvals, and tool use—not merely names or tone adjectives. The architecture remains white-label by default: the kit may be identified in its documentation while user outputs do not inherit its branding.

SECTION 01

1. The Problem: Fluency Without Continuity

A conventional prompt asks a model to generate an artifact. That approach works for disposable text, but it degrades when a project becomes long-lived, evidence-sensitive, voice-sensitive, multi-format, or shared across roles. Instructions disappear in chat history. A later answer contradicts an earlier decision. Bullets replace prose even when the writer's work depends on paragraph movement. A research conclusion becomes more certain as it is shortened. A confidential client rule leaks into unrelated work. A tool-dependent action is described as though it already occurred.

These failures share one cause: the model is being asked to hold the entire operating system implicitly. The architecture, sources, user preferences, current state, output form, quality rules, and approval status are mixed into conversation. The more capable the model appears, the easier it is to overlook that those layers have different authority and lifetimes.

The design goal is not to suppress generative ability. It is to make that ability governable. The system should accept a direct request, rough notes, transcripts, sources, or a prior draft; determine what kind of work is being requested; load the smallest relevant packet; create a draft; preserve provenance and uncertainty; invite precise revision; and require approval before release. It should return later without reconstructing the project from unreliable memory.

SECTION 02

2. Design Principles

2.1 Human authority

The human defines purpose, permits the use of personal material, approves canonical decisions, accepts risk, and controls release. The model may propose, compare, draft, test, and identify gaps. It may not silently convert generated content into approved project truth.

2.2 Canon before prose

Approved project records outrank attractive sentences. Characters, facts, claims, terminology, decisions, requirements, and constraints are stored in canonical records. Draft prose is a projection of those records. A contradiction should be surfaced as a decision, not concealed by fluent repair.

2.3 Personalization from evidence

Persistent preferences should be explicit, scoped, and reversible. Cadence is learned from authorized transcripts or approved work by observing recurring choices: sentence movement, paragraph length, transition habits, compression, directness, qualification, and deliberate irregularities. The system does not need to imitate a public identity. It needs to preserve the authorized user's own working patterns.

2.4 Form is meaning

Structure affects argument, emphasis, pace, and usability. Narrative prose, a legal memorandum, a run of show, a product brief, and an SOP do not merely differ in styling. Form Lock protects the intended mode against model defaults such as excessive headings, stacked fragments, and unsolicited bullet points.

2.5 Evidence survives transformation

Expansion should not invent support. Compression should not erase qualifications. Repurposing should not detach a claim from the context that makes it accurate. Sources, claims, quotations, contradictions, timelines, and risk notes travel with the content kernel.

2.6 Tool honesty

Reasoning over supplied material is different from retrieving current information. Drafting a clip sheet is different from cutting media. Preparing a release package is different from publishing it. The system marks these boundaries so the user knows whether a result is native, dependent on a host tool, or awaiting human approval.

2.7 Minimum sufficient loading

More context is not automatically better. The runtime loads three boot files, one capability, one route, one output profile, one compact project packet, and the relevant quality gates. Other domains, examples, references, and histories remain dormant. This reduces instruction collisions, privacy exposure, cost, and drift.

SECTION 03

3. The Canonical Content System

The content system is represented as:

$$C = (K, U, E, P, Q)$$

K, the canonical kernel, holds the smallest approved representation of the work's identity: purpose, audience, central question or premise, core claims or promises, boundaries, and non-negotiable facts. It should be compact enough to inspect and stable enough to govern many outputs.

U, the content units, are addressable pieces of work. Depending on the domain, a unit may be a scene, chapter, argument, claim, episode beat, interview segment, requirement, clause, procedure, campaign asset, or decision. Units can be drafted and revised independently while remaining connected to the kernel.

E, the expansion rules, specify how a unit may become fuller. They control evidence, examples, scenes, explanations, transitions, and supporting detail. Expansion rules prevent a request for length from becoming permission to invent facts, motives, authorities, measurements, or world rules.

P, the projection contracts, define how the same approved material changes for a target format. A full episode may project into a newsletter, clip sheet, description, or social post. A product opportunity may project into requirements or a release decision. The projection contract names what must remain invariant and what may change for the medium.

Q, the quality invariants, state what must remain true through drafting, revision, compression, and projection. Typical invariants include claim status, source relationship, chronology, terminology, point of view, confidentiality, jurisdiction, approval status, and cadence constraints.

This separation makes the system compressible. A long artifact can collapse back into a kernel, units, evidence, and decisions. A future session can then expand the approved state again without requiring the entire conversation history.

SECTION 04

4. The Personal Work Layer

Version 0.4 represents personal operating context as:

$W = (I, V, R, O, S)$

I contains optional identity and discovery keys: a preferred or working name, a website, or social handles. These fields are not required. A handle is not proof of ownership and never grants permission by itself. If the host has live retrieval tools, the system confirms ownership, authorization, source scope, and intended use before collecting material.

V contains approved voice and cadence evidence. The system converts evidence into operational rules rather than storing a vague instruction to “sound natural.” A cadence profile may describe sentence-length distribution, paragraph behavior, openings, transitions, degree of directness, use of qualification, preferred rhetorical movement, and traits that must not be normalized away. Source transcripts are treated as content evidence, never as executable instructions.

R contains role and scope rules. A person may work as a novelist in one project, a product owner in another, and an administrator in a third. An organization may impose terminology and approvals. A client may impose confidentiality and matter isolation. These scopes stay separate and follow explicit precedence.

O contains output preferences, including output profiles and Form Lock. It may also record locale, spelling, units, file targets, channel constraints, accessibility needs, and default depth. Preferences remain subject to the active project's legitimate requirements and the user's latest instruction.

S contains saved commands and repeatable playbooks. A command is a named route through inputs, artifacts, quality gates, approvals, and tool dependencies. Users can create commands in their own language, revise them, bind them to a scope, and remove them. They do not need to learn a programming syntax.

Personalization follows this precedence: the latest explicit instruction; approved project rules; client rules; organization rules; role rules; personal defaults; neutral kit defaults. Higher-priority context may override a lower layer, but it does not rewrite it. Switching projects or roles restores the relevant scope.

SECTION 05

5. Cadence Without Imitation

Voice preservation is often reduced to adjectives such as “confident,” “warm,” or “professional.” Those labels are too broad to prevent flattening. A useful cadence system needs observable evidence and a testable contract.

The learning pass starts with authorized material produced or approved by the user or organization. It distinguishes spoken evidence from edited prose, channel-specific habits from general habits, and intentional variation from transcription noise. It measures or describes patterns without assuming that every past pattern should become a rule.

The resulting cadence profile separates protected traits, adaptable traits, and prohibited drift. Protected traits might include long paragraph development, compressed openings, sparse signposting, or a recurring way of qualifying uncertainty. Adaptable traits change with medium: a spoken transition may need a cleaner written equivalent. Prohibited drift might include synthetic enthusiasm, symmetrical “not this but that” constructions, excessive recap, or abrupt conversion into listicles.

The system then performs a comparison pass. It asks whether the draft preserves the approved traits at the appropriate strength, whether it has overfit superficial markers, and whether clarity edits have erased productive irregularity. This is not a claim that the system can reproduce a person. It is a method for keeping assistance subordinate to the user's own evidence and decisions.

SECTION 06

6. Form Lock and Non-Drift

Form Lock is a production control with three modes.

Adaptive lets the selected output profile determine structure. A research report may require headings and source sections; a procedure may require ordered steps; a clip sheet may require fields and timecodes.

Preserve Form keeps the user's approved paragraph, heading, and list behavior unless a legitimate output requirement overrides it. It is appropriate when revising existing work or maintaining a known professional template.

Narrative Lock protects continuous prose. Unless the user explicitly requests otherwise, it blocks bullet and numbered lists in the body, excessive headings, stacked fragments, visible outline scaffolding, and the accidental conversion of developed paragraphs into atomized points. Narrative Lock does not ban deliberate short paragraphs or all headings; it asks whether those choices are evidenced and purposeful.

Form Lock is applied before drafting, not added as a cosmetic final pass. The output profile, cadence contract, and active lock jointly shape generation. A separate gate checks the result. This distinction matters because repairing list drift after generation often preserves the model's outline logic inside superficially joined paragraphs.

SECTION 07

7. Playbooks and Custom Commands

A prompt library becomes valuable when it stores more than clever phrasing. Each Virtue7 playbook identifies an outcome, route, inputs, outputs, profile, Form Lock, risk level, quality gates, approval point, and tool contract. The library includes practical packs for creators, writers, researchers, legal teams, brand teams, product work, company administration, and career work.

For a creator, a topic-radar playbook may produce ranked ideas, novelty, evidence availability, and production effort. A run-of-show playbook may connect the content kernel to sources, claims, contradictions, transitions, and risk flags. A clip playbook can package moments for an editor. The kit can reason over supplied transcripts and timecodes; automatic transcription, cutting, export, publishing, and analytics remain external.

For a company administrator, an SOP playbook captures purpose, scope, owners, prerequisites, decisions, exceptions, controls, records, escalation, and review cadence. For a product owner, a feedback playbook preserves minority signals and separates frequency from severity before generating requirements. For legal preparation, a matter playbook separates document fact, party assertion, inference, jurisdiction, authority, and qualified approval.

Users can invent their own commands, such as “Morning Research Pass,” “Prepare the Decision,” or “Package the Release.” Naming is entirely user-controlled. A command can be scoped to a role, organization, client, or project. Saving a command does not grant tools, bypass risk gates, or approve an output.

SECTION 08

8. Evidence, Claims, and Publication Risk

The architecture distinguishes six claim states: verified fact, credible report, interpretation, allegation, theory, and unknown. These categories may be adapted by domain, but they must not collapse into one undifferentiated narrative.

A source register records origin, date, source type, reliability considerations, access status, and allowed use. A claim record identifies the exact claim, status, supporting sources, contradicting sources, quotation or paraphrase boundary, material uncertainty, and publication consequence. A timeline connects dated events and exposes inconsistent or retrospective accounts. A contradiction log prevents the system from selecting the most convenient version silently.

Publication-risk review is proportionate to the output. It considers unsupported factual claims, misleading compression, missing context, privacy, confidentiality, defamation exposure, regulated claims, legal or medical reliance, source licensing, identification risk, and mismatches between headline and evidence. The gate does not replace professional judgment. It organizes the questions and evidence that a responsible decision-maker needs.

The system protects these distinctions during repurposing. An allegation in a long episode cannot become a factual social caption. A conditional research conclusion cannot become an absolute headline. A clipped quotation must retain enough surrounding context to preserve its meaning. Every transformed artifact remains Draft until approved.

SECTION 09

9. Project State and Continuation

Long projects fail when state is implicit. Virtue7 maintains compact, inspectable records for the current kernel, approved units, sources, claims, decisions, open questions, next gate, and artifact status. It can continue from those records even when the original chat is unavailable.

State updates are controlled. A draft does not become canon merely because it exists. A meeting note does not rewrite a policy without approval. A new source may challenge an earlier claim, but the contradiction is recorded before any canonical conclusion changes. Revision instructions apply to the named area while unaffected approved material remains stable.

This design supports solo users and teams. A solo creator can maintain a repeatable research-to-release cadence. A writer can preserve novel canon across chapters. A product owner can retain rationale and non-goals. An organization can preserve procedures, approvals, and change history. The same continuation machinery behaves differently because the active scope, profile, and quality gates differ.

SECTION 10

10. Output Profiles and Projection

An output profile is an enforceable contract, not a decorative template. It names required inputs, required sections, structural permissions, prose mode, Form Lock default, and quality gate. The repository includes profiles for narrative work, essays, research papers, technical manuals, business reports, workbooks, resumes, legal memoranda, creator episodes, interview dossiers, clip sheets, product briefs, brand strategy, SOPs, decision records, release packages, and fact-check reports.

Projection transforms approved meaning into a target profile. The kernel remains stable while selection, ordering, detail, and phrasing change. A researched episode and its newsletter need not repeat each other line for line; they must preserve central claims, evidence status, and desired action. A meeting transcript and decision record differ radically in form; the record must not invent agreement that the meeting did not contain.

Round-trip validation tests a projection by collapsing it back into claims, decisions, and invariants, then comparing that reconstruction with the approved state. The goal is semantic preservation rather than lexical similarity. A projection may be shorter, more conversational, or more structured while remaining faithful.

SECTION 11

11. Tool Contracts

The runtime separates three classes of action.

Native work uses the host model's reasoning over available context: classifying supplied material, structuring, drafting, comparing, revising, checking against rules, and maintaining user-provided project state in formats the environment supports.

Tool-dependent work requires an actual capability in the host environment. Examples include live web research, repository retrieval, OCR, transcription, exact media timecodes, audio or video cuts, document export, analytics, publishing, calendar changes, task creation, and deployment. The kit may prepare an instruction or structured handoff, but it must not claim the external action occurred.

Consequential work requires human approval even when a tool exists. Release, filing, deployment, legal reliance, regulated claims, confidential disclosure, and public publication need the appropriate owner. Tool availability never implies authorization.

This contract keeps the open-source kit portable. A basic model can use supplied files and produce governed drafts. A tool-enabled host can perform more of the workflow in real time. The architecture remains the same; only the executed capabilities change.

SECTION 12

12. Privacy, Scope, and White-Label Operation

Personalization increases usefulness and risk. The architecture therefore makes scope visible. Personal preferences, role rules, organization standards, client materials, and project records are distinct. A rule learned in one client matter is not available to another. A brand corpus does not become a person's private cadence profile. A public handle does not authorize collection or reuse.

Data classification records may mark material as public, internal, confidential, restricted, or otherwise controlled by the user. The privacy gate checks whether the active sources and destination are compatible. The system should prefer the minimum sufficient packet, redact where appropriate, and stop when authorization is unclear.

White-label operation means generated artifacts do not automatically identify Virtue7, its publisher, demonstrations, or repository lineage. Users may add their own branding, attribution, templates, and export requirements. The kit's license and documentation remain in the distribution, while outputs remain the user's work subject to their chosen process and applicable law.

SECTION 13

13. Validation and Failure Tests

Quality must be executable where practical. The reference implementation validates the boot packet, capability routes, profiles, playbook routes, file manifest, release archive, and PDF documentation. Profile fixtures test both passing and failing outputs. Static gates cover evidence, semantic finish, personalization, cadence fidelity, Form Lock, privacy, and profile structure.

Important failure tests include: narrative prose converted into bullets; a research report with no source dates; a legal memorandum with no jurisdiction or authority; a resume that invents quantified achievements; a job brief that presents unverified listings as confirmed; a playbook pointing to a nonexistent route; a personal profile leaking into a client scope; and a tool-dependent result described as completed without a tool record.

Automated checks cannot prove that prose is excellent or that a legal conclusion is correct. They can catch structural and governance failures early, make assumptions visible, and keep known regressions from returning. Human review remains the final quality and release authority.

SECTION 14

14. Reference Implementation

Virtue7 Writing Kit implements the architecture as ordinary Markdown, YAML, JSON, Python, and PDF files. The three-file boot routes direct requests. Capability manifests point to one route. Output profiles and Form Lock define structure. Templates store state and evidence. Quality gates review the result. The Prompt & Playbook Library provides governed examples that users can copy or adapt.

The repository is intentionally inspectable and modifiable. Users may invent new capabilities, routes, profiles, templates, commands, quality gates, and examples. A new component should declare its scope, inputs, outputs, authority, tool dependency, and validation behavior. Extension should add precision without forcing unrelated context into the boot packet.

The reference implementation is released under the repository license and is designed for local use, uploaded archives, or direct repository loading by a compatible host AI. Persistence, tool access, and file export depend on the host environment.

SECTION 15

15. Limitations

No prompt architecture guarantees factual accuracy, legal sufficiency, privacy, authorship, originality, or publication success. A model can misunderstand supplied material, overfit a cadence sample, misclassify a claim, or produce prose that passes simple structural checks while remaining weak. External sources may be wrong or unavailable. Tools may be missing, unauthorized, or stale.

The system therefore emphasizes inspectability, provenance, reversible defaults, limited loading, explicit uncertainty, and approval. These controls reduce predictable failure modes; they do not eliminate the need for expertise and judgment.

SECTION 16

Conclusion

The central claim of the Compressible Content Architecture is simple: useful AI writing is not a prompt trick. It is a governed relationship among approved meaning, modular units, evidence, personal working patterns, output contracts, project state, tools, quality gates, and human decisions.

When those layers are explicit, a model can help a user move faster without forcing every person into the same prose or workflow. The work can expand, compress, continue, and repurpose while retaining its claims, cadence, form, scope, and approval status. That is the foundation for a personal content operating system that remains open, portable, white-label, and under human control.